Analysis of Neural-Network-Based Convex Regularizer for Image Reconstruction

Sullivan Castro
ENS Paris-Saclay MVA
sullivan.castro@eleves.enpc.fr

Matthieu Mérigot-Lombard
ENS Paris-Saclay MVA
merigotmatthieu@gmail.com

1 Introduction

Inverse problems arise in many scientific and engineering disciplines, where the goal is to retrieve the inputs $\boldsymbol{x} \in \mathbb{R}^d$ from altered and noisy observations:

$$\boldsymbol{y} = \boldsymbol{H}\boldsymbol{x} + \boldsymbol{\epsilon} \in \mathbb{R}^m$$

where $\boldsymbol{H} \in \mathbb{R}^{m \times d}$ and $\boldsymbol{\epsilon} \in \mathbb{R}^m$ represents the noise. Given $\boldsymbol{y}$, the objective is to reconstruct $\boldsymbol{x}$. This is a classical problem in fields such as medical imaging [1] or finance [2]. Often, the problem is ill-posed with only a few mixed observations ($m < d$), which can make the reconstruction more challenging. The classical reconstruction is computed as:

$$\boldsymbol{x}^* \in \arg\min_{\boldsymbol{x} \in \mathbb{R}^d} \|\boldsymbol{H}\boldsymbol{x} - \boldsymbol{y}\|_2^2 + \lambda R(\boldsymbol{x}) \tag{1}$$

where R is a convex regularizer that incorporates prior information about $\boldsymbol{x}$, which can be for example a Tikhonov regularizer [3] or a total-variation regularizer [4].

To retrieve the reconstructed inputs, deep learning-based methods have become increasingly popular [5], [6]. These approaches are often much faster and provide significantly improved quality of reconstructions compared to classical methods. However, the main disadvantage of deep learning-based approaches is that the neural network does not offer control over the data-consistency term $\|\boldsymbol{H}\boldsymbol{x} - \boldsymbol{y}\|_2^2$. Moreover, these models are not universal, as each model is specifically trained for a given $\boldsymbol{H}$ and ϵ.

Another solution is the Convolutional Neural Network (CNN) variants of the Plug-and-Play (PnP) framework [7], [8], [9], which view the denoising task as:

$$\boldsymbol{x}^* \in \arg\min_{\boldsymbol{x} \in \mathbb{R}^d} \|\boldsymbol{x} - \boldsymbol{y}\|_2^2 + \lambda R(\boldsymbol{x}) = \text{prox}_R^{\lambda}(\boldsymbol{y}) \tag{2}$$

The method presented here (CRR-NN) aims to optimize the same problem. However, instead of directly training an optimal denoiser $D_\sigma^* \sim \text{prox}_{\lambda R^*}^{\tau}$, it focuses on learning an optimal regularizer R^* with respect to Equation 2. This is achieved using an interpretable and expressive model that is very efficient to train.

2 Regularizer Construction

2.1 Convex-Ridge Regularizer

Definition 2.1 (Convex-Ridge Regularizers)**.** *A convex-ridge function is defined as:*

$$R : \boldsymbol{x} \mapsto \sum_i \psi_i(\boldsymbol{w_i}^T \boldsymbol{x})$$

where the profile functions ψ_i are convex and possess Lipschitz-continuous derivatives, while $w_i \in \mathbb{R}^d$ are learnable weights.

From now on, the notation R_θ denotes the regularizer R parameterized by the learnable set of parameters θ.

Property 1. *The gradient of R_θ can be expressed as:*

$$\nabla R_\theta(\boldsymbol{x}) = \boldsymbol{W}^T \sigma(\boldsymbol{W}\boldsymbol{x})$$

where $\boldsymbol{W} = [\boldsymbol{w_1} \dots \boldsymbol{w_p}]^T \in \mathbb{R}^{p \times d}$ *and* σ *is a pointwise activation function whose components* $(\sigma_i = \psi_i')_{i=1}^p$ *are Lipschitz-continuous.*

The choice of the activation functions σ_i is determinant in the expressivity of the learned regularizer R_θ. The optimal choice should allow for sufficient flexibility while maintaining convexity and stability.

Definition 2.2 (Linear Spline Activation Function)**.** *A linear spline activation function* $\sigma : \mathbb{R} \to \mathbb{R}$ *is a piecewise-linear function defined by a set of knots* (x_k, v_k) *such that:*

$$\sigma(x) = v_k + s_k(x - x_k), \quad \forall x_k \leq x < x_{k+1}$$

where s_k *are the slopes between adjacent knots. Let* $\mathcal{LS}_\uparrow^m$ *denotes the subset of increasing linear splines with at most* m *knots.*

Theorem 2.1. *Let* $\Omega \subset \mathbb{R}^d$ *be compact. The set of functions* R_θ *with gradients:*

$$\{\boldsymbol{W}^T \sigma(\boldsymbol{W}\cdot) | \boldsymbol{W} \in \mathbb{R}^{p \times d}, \sigma_i \in \mathcal{LS}_\uparrow^m\} \tag{3}$$

is dense in the space of scalar Lipschitz-continuous and increasing functions on Ω *with respect to* $\|\cdot\|_{C(\Omega)}$.

Corollary 1. *The set of functions for which the gradient is in Equation 3 is dense in the space of convex-ridge functions with respect to* $\|\cdot\|_{C^1(\Omega)}$.

Property 2. *The set of functions:*

$$\{\boldsymbol{W}^T \mathit{ReLU}(\boldsymbol{W} \cdot - b) | \boldsymbol{W} \in \mathbb{R}^{p \times d}, b \in \mathbb{R}^p\}$$

is ***not*** *in the space of scalar Lipschitz-continuous and increasing functions on* Ω *with respect to* $\|\cdot\|_{C(\Omega)}$.

This implies that ReLU-based models have limited expressivity compared to spline-based models which cover the full range of convex-ridge functions.

2.2 Stability

Property 3. *Let* $\boldsymbol{H} \in \mathbb{R}^{m \times d}$ *and* $\psi_i : \mathbb{R} \to \mathbb{R}$, $i = 1, \dots, p$ *be convex functions. If arg* $\min_{t \in \mathbb{R}} \psi_i(t) \neq \emptyset$, *for all* $i = 1, \dots, p$, *then:*

$$\emptyset \neq \mathit{arg}\min_{\boldsymbol{x} \in \mathbb{R}^d} \frac{1}{2}\|\boldsymbol{Hx} - \boldsymbol{y}\|_2^2 + \sum_{i=1}^p \psi_i(\boldsymbol{w_i}^T \boldsymbol{x})$$

Property 4. *Let* $\boldsymbol{H} \in \mathbb{R}^{m \times d}$ *and* $\psi_i : \mathbb{R} \to \mathbb{R}$, $i = 1, \dots, p$ *be convex continuously differentiable functions with arg* $\min_{t \in \mathbb{R}} \psi_i(t) \neq \emptyset$. *For any* $\boldsymbol{y_1}, \boldsymbol{y_2} \in \mathbb{R}^m$, *let:*

$$\boldsymbol{x_q} \in \mathit{arg}\min_{\boldsymbol{x} \in \mathbb{R}^d} \frac{1}{2}\|\boldsymbol{Hx} - \boldsymbol{y}\|_2^2 + \sum_{i=1}^p \psi_i(\boldsymbol{w_i}^T \boldsymbol{x})$$

with $q = 1, 2$ *be the corresponding reconstructions. Then:*

$$\|\boldsymbol{Hx_1} - \boldsymbol{Hx_2}\|_2 \leq \|\boldsymbol{y_1} - \boldsymbol{y_2}\|_2$$

Proof 2.1 (Property 4)**.** *Property 3 guarantees the existence of* $\boldsymbol{x_q} \in \mathit{arg}\min_{\boldsymbol{x} \in \mathbb{R}^d} \frac{1}{2}\|\boldsymbol{Hx} - \boldsymbol{y}\|_2^2 + \sum_{i=1}^p \psi_i(\boldsymbol{w_i}^T \boldsymbol{x})$.

Since the minimization problem is unconstrained, $\boldsymbol{x_q}$ *cancels the gradient of the objective function. Let* $\boldsymbol{x_1}, \boldsymbol{x_2}$ *be the reconstructions of* $\boldsymbol{y_1}, \boldsymbol{y_2}$, *so:*

$$\begin{aligned}
& H^T(Hx_q - y_q) + \nabla R(x_q) = 0 \\
\implies & H^T H(x_1 - x_2) + (\nabla R(x_1) - \nabla R(x_2)) = H^T(y_1 - y_2) \\
((x_1 - x_2)^T \times) \implies & \|Hx_1 - Hx_2\|_2^2 + (x_1 - x_2)^T(\nabla R(x_1) - \nabla R(x_2)) = (H(x_1 - x_2))^T(y_1 - y_2)
\end{aligned}$$

Since ∇R *is convex, for all* x_1, x_2, *we have* $(x_1 - x_2)^T(\nabla R(x_1) - \nabla R(x_2)) \geq 0$. *Therefore:*

$$\begin{aligned}
& \|Hx_1 - Hx_2\|_2^2 \leq (H(x_1 - x_2))^T(y_1 - y_2) \\
(\textit{Cauchy-Schwarz inequality}) \implies & \|Hx_1 - Hx_2\|_2^2 \leq \|Hx_1 - Hx_2\|\|y_1 - y_2\| \\
\implies & \|Hx_1 - Hx_2\|_2 \leq \|y_1 - y_2\|
\end{aligned}$$

Thus, if the perturbations $\boldsymbol{y_1}$ and $\boldsymbol{y_2}$ are close, the reconstructions $\boldsymbol{Hx_1}$ and $\boldsymbol{Hx_2}$ will also be close. So, if the perturbation $\boldsymbol{y}$ changes "slightly," the optimal reconstruction $\boldsymbol{x^*}$ will not diverge. This is a fundamental property in image reconstruction, helping to avoid inconsistent reconstructions that amplify noise.

3 Regularizer Training

3.1 Gradient-Step and Convergence

Definition 3.1 (Gradient-Step)**.** *Given the assumption on R_θ for the denoising task defined in* (2)*, the gradient step can be expressed as:*

$$\begin{aligned} T_{R_\theta,\lambda,\alpha} &= \boldsymbol{x} - \alpha\left((\boldsymbol{x}-\boldsymbol{y}) + \lambda\nabla R_\theta(\boldsymbol{x})\right) \\ &= (1-\alpha)\boldsymbol{x} + \alpha\left(\boldsymbol{y} - \lambda \boldsymbol{W}^T\sigma(\boldsymbol{W}\boldsymbol{x})\right) \end{aligned}$$

This formulation can be interpreted as a one-hidden-layer CNN with a bias and a skip connection.

Theorem 3.1. *The t-step denoiser $T^t_{R_\theta,\lambda,\alpha}$, defined as the operator obtained by composing t gradient-step operators, is contractive for $\alpha \in \left[0, \frac{2}{2+\lambda L_\theta}\right]$, where L_θ is the Lipschitz constant of ∇R_θ. Under these conditions, the gradient descent method is guaranteed to converge.*

Proof 3.1 (Theorem 3.1)**.** *Define the operator:*

$$T(\boldsymbol{x},\boldsymbol{y}) = \boldsymbol{x} - \alpha\left((\boldsymbol{x}-\boldsymbol{y}) + \lambda\nabla R_\theta(\boldsymbol{x})\right)$$

Let $\boldsymbol{x}_0 = \boldsymbol{y}$ and define the iterative sequence:

$$\boldsymbol{x_{k+1}} = T(\boldsymbol{x_k},\boldsymbol{y})$$

Define the mapping $L_k : \boldsymbol{y} \mapsto \boldsymbol{x_k}$, then it holds that:

$$L_{k+1} = U \circ L_k + \alpha\, Id$$

where $U = Id - \alpha(Id + \lambda\nabla R_\theta)$.

The Jacobian of U is given by:

$$J_U = I - \alpha(I + \lambda H_{R_\theta})$$

and satisfies:

$$((1-\alpha) - \alpha\lambda L_\theta)I \preceq J_U \preceq (1-\alpha)I$$

Thus, the Lipschitz constant of U satisfies:

$$\mathit{Lip}(U) \leq \max(\alpha\lambda L_\theta - (1-\alpha), 1-\alpha)$$

Therefore, if $\alpha \leq \frac{2}{2+\lambda L_\theta}$, then:

$$\mathit{Lip}(U) \leq 1 - \alpha < 1$$

This ensures the contractivity of the t-step denoiser and guarantees the convergence of the gradient descent method.

3.2 Multi-Gradient-Step Denoiser Training

Let $\{\boldsymbol{x}^m\}_{m=1}^M$ be a set of clean images and $\{\boldsymbol{y}^m = \boldsymbol{x}^m + \boldsymbol{\epsilon}^m\}_{m=1}^M$ their corresponding noisy versions, where $\boldsymbol{\epsilon}^m$ represents the noise realizations. Let $\mathcal{L}$ be a loss function. The optimal parameters are determined by minimizing the loss:

$$\boldsymbol{\theta}_t^*, \boldsymbol{\lambda}_t^* \in arg\min_{\theta,\lambda} \sum_{m=1}^{M} \mathcal{L}(T^t_{R_\theta,\lambda,\alpha}(\boldsymbol{y}^m), \boldsymbol{x}^m)$$

The objective of this optimization problem is to find the proximal operator that minimizes the loss. It has been previously shown that, for a suitable choice of α, the gradient-step iteration converges, ensuring that $T^\infty_{R_\theta,\lambda,\alpha} = \text{prox}^\lambda_{R_\theta}$. However, to reduce training time, an approximation of the proximal operator is used by limiting the number of gradient steps to t. Therefore, optimization could be performed using only a single gradient step, referred to as a mono-gradient-step approach. However, employing a limited number of steps reduces the accuracy of the proximal operator. But, this does not necessarily imply a degradation in the overall performance of the final algorithm.

In fact, the results presented in Figure 1 and Table 1 suggest that approximating the proximal operator with a finite number of steps is sufficient for learning the regularizer R_θ while achieving even better performance. This indicates that a faster training procedure can yield improved results. Specifically, the PSNR for the CRR-NN (t-step) model, optimized with an optimal

number of steps, reaches 36.97 dB for $\sigma = 5/255$ and 28.12 dB for $\sigma = 25/255$, while the CRR-NN (proximal) achieves slightly lower values of 36.96 dB and 28.11 dB, respectively.

The experimental results also highlight the advantages of a multi-gradient-step approach over a mono-gradient-step approach. As shown in Figure 1, the CRR-NN (t-step) model performance slightly increases with larger values of t: from ∼36.85 dB to 36.97 dB for $\sigma = 5/255$, and from ∼27.82 dB to 28.12 dB for $\sigma = 25/255$. This confirms that applying multiple gradient steps enhances the regularization process, leading to improved reconstruction quality. However, the PSNR gain saturates after approximately 10 gradient-steps, indicating that increasing t does not necessarily lead to further improvements.

	$\sigma = \frac{5}{255}$	$\sigma = \frac{25}{255}$
TV*,‡	36.41	27.48
Higher-order MRFs*,‡	NA	28.04
VN$^{\dagger}_{1,t}$	NA	27.69
βCNN$^{\ddagger}_{\sigma}$	36.48	27.69
D$^{\ddagger}_{\text{ISTA}}$	36.54	NA
GS-DnICNN†	36.85	27.76
D$^{\ddagger}_{\text{ADMM}}$	36.62	NA
CRR-NN-ReLU (t-step)†,‡	35.50	26.75
CRR-NN (t-step)†,‡	**36.97**	**28.12**
CRR-NN (proximal)*,‡	36.96	28.11

* Full minimization of a convex function
† Partial minimization of a convex function
‡ Stable steps (averaged layers)

Table 1: Evaluation of convex models and averaged denoisers on BSD68 [10]

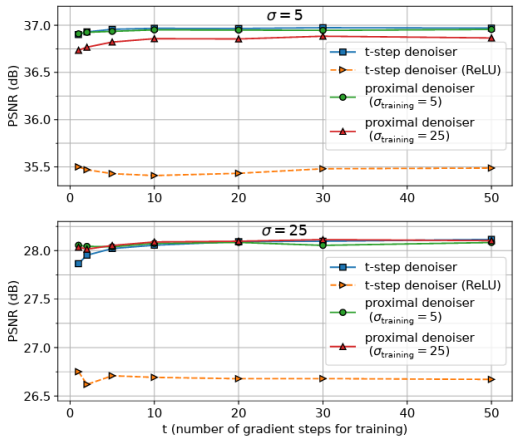

Figure 1: Denoising performance of CRR-NNs for noise level $\sigma = 5/255$ and $\sigma = 25/255$ versus the number of gradient steps used for training on BSD68 [10]

Another point to mention is that the CRR-NN model also performs well for noise levels σ_{training} that differ from the real Gaussian noise level σ. Figure 1 shows that using the CRR-NN model (proximal) with $\sigma_{\text{training}} = 25/255$ on $\sigma = 5/255$ results in a difference of 0.20 dB, while using the CRR-NN model (proximal) with $\sigma_{\text{training}} = 5/255$ on $\sigma = 25/255$ shows almost no difference. This implies that the CRR-NN model trained on low levels of noise remains highly effective for higher levels of noise.

3.3 Enforcing Constraints

In order to obtain the properties of expressivity, stability, and convergence mentioned earlier, we need to enforce certain constraints on σ_i during training. These constraints should ensure that σ_i is increasing (the convexity constraint on ψ_i), that σ_i takes the value 0 somewhere (the existence constraint), and that the step size α guarantees convergence.

The specifics will not be discussed here. In short, the first two constraints are enforced using a regularization term in the training loss, while the third constraint computes an upper bound on the Lipschitz constant L_θ during the forward pass to select the appropriate step size α.

4 Initial Image Recovery

The regularizer R_θ is now considered trained. Before reconstruction, CRR-NN introduces a new tunable hyperparameter $\mu \in \mathbb{R}^+$ the scaling parameter such that:

$$\boldsymbol{x}^* \in \underset{\boldsymbol{x}\in\mathbb{R}^d}{\arg\min} \|\boldsymbol{H}\boldsymbol{x} - \boldsymbol{y}\|_2^2 + \frac{\lambda}{\mu} R_\theta(\mu\boldsymbol{x}) \tag{4}$$

The new gradient step is then given by:

$$T_{R_\theta,\lambda,\mu,\alpha} = \boldsymbol{x} - \alpha\left(\mathbf{H}^T(\mathbf{H}\boldsymbol{x} - \boldsymbol{y}) + \lambda\nabla R_\theta(\mu\boldsymbol{x})\right)$$

with a new convergence condition:

$$\alpha \in \left[0, \frac{2}{2 + \lambda\mu L_\theta}\right]$$

Algorithm 1 FISTA

Require: $x_0 \in \mathbb{R}^d$, $y \in \mathbb{R}^m$, $\lambda \geq 0$, $\mu > 0$
Ensure: x_k
  Set $k \leftarrow 0$, $z_0 \leftarrow x_0$, $\alpha \leftarrow \frac{1}{\lambda\mu L_\theta + \|\mathbf{H}\|^2}$, $t_0 \leftarrow 1$
  while tolerance not reached **do**
    $x_{k+1} \leftarrow \big(z_k - \alpha\big(\mathbf{H}^T(\mathbf{H}z_k - y) + \lambda\nabla R(\mu z_k)\big)\big)_+$
    $t_{k+1} \leftarrow \frac{1+\sqrt{4t_k^2+1}}{2}$
    $z_{k+1} \leftarrow x_{k+1} + \frac{t_k - 1}{t_{k+1}}(x_{k+1} - x_k)$
    $k \leftarrow k + 1$
  end while
  return x_k

To solve Equation (4), CRR-NN uses the Fast Iterative Shrinkage-Thresholding Algorithm [11] (FISTA), which is detailed in Algorithm 1.

5 Comparison with PnP-PGD/RED-PGD and Results

PnP-PGD

Let $f(\boldsymbol{x}) = \|\boldsymbol{H}\boldsymbol{x} - \boldsymbol{y}\|_2^2$ be the data fidelity term. The Plug-and-Play Proximal Gradient Descent (PnP-PGD) is a resolution method for the PnP framework [7] which is widely used for solving inverse problems by iteratively refining an estimate of the initial signal. Instead of using an explicitly defined proximal operator associated with a hand-crafted regularizer like the CRR-NN algorithm, PnP methods replace it with a learned or pre-trained denoiser, leveraging the prior knowledge embedded in deep neural networks such that:

$$D_\sigma \sim \text{prox}^\tau_{\lambda R^*}$$

Then, the PGD algorithm alternates between a gradient descent step on the data-fidelity term and an implicit regularization step, where the denoiser acts as a surrogate for the proximal operator:

$$\boldsymbol{x}_{k+1} = D_\sigma \circ (I - \tau\nabla f)(\boldsymbol{x}_k)$$

If f is convex, differentiable with ∇f L-Lipschitz, and D_σ is θ-averaged with $\theta \in (0, 1)$, it can be shown that this algorithm converges for $\tau L < 2$. This allows the reconstruction process to integrate learned priors without requiring an explicit formulation of the regularizer. Also, unlike end-to-end deep learning models, which lack explicit control over data consistency, PnP methods ensure that the estimated reconstruction remains close to the observed measurements.

RED-PGD

The Regularization by Denoising Proximal Gradient Descent [12] (RED-PGD) follows a different approach from PnP: instead of performing a gradient step on f and a proximal step on the regularizer, it reverses the order, leading to the iterative update:

$$\boldsymbol{x}_{k+1} = \text{prox}^\tau_f \circ (I - \tau\lambda\nabla g_\sigma)(\boldsymbol{x}_k)$$

where the denoiser is trained to satisfy the relation:

$$D_\sigma = I - \nabla g_\sigma \sim I - \nabla R^*$$

Beyond the iterative scheme, RED-PGD shares similarities with CRR-NN, as both methods aim to learn an optimal regularizer. However, they differ in how the best regularizer is defined: RED-PGD seeks to minimize the loss using a regularizer that solves Equation 1, while CRR-NN optimizes based on Equation 2.

Let $F = f + \lambda g_\sigma$ and L be the Lipschitz constant of ∇g_σ. A sufficient condition for the convergence of PGD is:

$$\forall \tau < \frac{1}{L}, \quad F(x_k) - F(T_\tau(x_k)) \geq \left(\frac{1}{2\tau} - \frac{L}{2}\right)\|T_\tau(x_k) - x_k\|^2$$

Since estimating the Lipschitz constant L can be difficult, a backtracking procedure is introduced to adaptively determine τ. Given parameters $\gamma \in (0, \frac{1}{2})$, $\eta \in [0, 1)$, the procedure is defined as:

$$\textbf{while} \quad F(x_k) - F(T_\tau(x_k)) < \frac{\gamma}{\tau}\|T_\tau(x_k) - x_k\|^2 \quad \textbf{do} \quad \tau \leftarrow \eta\tau$$

Since the stopping criterion ensures that the inequality does not hold for $\tau < \frac{1-2\gamma}{L}$, the backtracking loop terminates in a finite number of iterations. Moreover, it can be shown that the convergence guarantees of the algorithm remain valid when backtracking is employed.

Results

The results of the PnP-PGD, RED-PGD, and CRR-NN algorithms with pretrained models for an image denoising task are presented in Figure 2. The noise degradation process was performed using the blur kernel `kernel8.txt` from TP8-PnP of size 25×25 and additive Gaussian noise with $\sigma = \frac{1}{255}$. The λ, μ hyperparameters from CRR-NN were determined through a grid search. The hyperparameters of PnP and RED were the same than the ones used in TP9. However, the choice of σ_{training} and t for CRR-NN is constrained by the selection of pretrained models. But, as previously observed, the model remains effective for noise levels different from those used during training, and its performance is relatively insensitive to variations in t. The experiments were conducted on an RTX 3080 Ti running Linux 6.12.19-1-lts with CUDA 12.8.

Figure 2: Results of image denoising task using PnP-PGD, RED-PGD, and CRR-NN algorithms with pretrained models after $n = 100$ iterations of each iterative algorithm. The noise degradation process was done using a blur kernel of size 25×25 and a Gaussian noise with $\sigma = 1/255$. PnP-PGP uses $\sigma_{\text{training}} = 2\sigma$ and $\tau = 1.9\sigma^2$. RED-PGP uses $\sigma_{\text{training}} = 1.8\sigma$, $\tau = 2.10^{-5}$, $\lambda = 1600$, $\gamma = 0.4$ and $\eta = 0.9$. CRR-NN uses $\sigma_{\text{training}} = 5$, $\lambda = 25$, $\mu = 4$ and $t = 10$.

The results indicate that all models achieve effective image denoising, with a Peak Signal-to-Noise Ratio (PSNR) greater than 28. Among them, RED-PGD produces the lowest-quality output with PSNR $= 28.96$, showing noticeable artifacts in the background, though these are less prominent in other regions of the image. PnP-PGD achieves a slightly better result with PSNR $= 30.56$, where similar artifacts remain but are less perceptible. Finally, CRR-NN demonstrates the best performance with PSNR $= 37.87$, significantly outperforming the other two models. The generated output is nearly indistinguishable from the original image, except for slight vertical lines on the right side of the camera.

Algorithm	Time (s)
PnP-PGD	1.6
RED-PGD	3.3
CRR-NN	0.9

Table 2: Computation time of PnP-PGD, RED-PGD, and CRR-NN

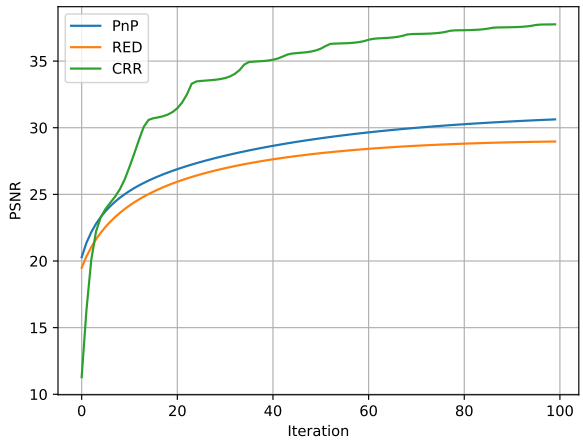

Figure 3: PSNR evolution of PnP-PGD, RED-PGD, and CRR-NN for the denoising task in Figure 2

Figure 3 shows the evolution of PSNR for the three algorithms. Initially, all curves shows an increase with the number of iterations, maintaining a positive slope even at the end of the 100 iterations. This suggests that more iterations could further enhance performance. Due to the sublinear growth, the algorithms rapidly achieve good performance, making further improvements increasingly difficult as iterations progress.

Moreover, there is a noticeable difference in behavior between the CRR-NN evolution and that of PnP-PGD and RED-PGD. Specifically, the learning curve of CRR-NN appears less stable, whereas the other two algorithms exhibit a smoother trend. The CRR-NN algorithm starts with a significantly lower PSNR but undergoes a greater improvement over iterations, allowing it to outperform the other methods in the final results presented in Figure 2.

Finally, Table 2 presents the computation time for each algorithm. Among them, CRR-NN is the fastest, requiring only 0.9 seconds, whereas PnP-PGD takes 1.6 seconds. In contrast, RED-PGD is the slowest, with a computation time of 3.3 seconds. These results highlight the efficiency of the CRR-NN method.

6 Conclusion

In this report, we explained the construction of a denoising method based on learning a convex-ridge regularizer that is both interpretable and expressive, while also being efficient in training, stable, and having good convergence properties.

Experimental results demonstrated that increasing the number of gradient steps enhances reconstruction quality, although the PSNR gain eventually saturates beyond a certain threshold. Moreover, comparisons with PnP-PGD and RED-PGD highlighted the advantages of the proposed method in learning an effective regularizer while preserving computational efficiency and ensuring convergence. This approach thus combines the benefits of purely deep learning-based methods while guaranteeing stability and efficiency.

References

[1] Yang Song, Liyue Shen, Lei Xing, and Stefano Ermon. Solving inverse problems in medical imaging with score-based generative models, 2022.

[2] Yasushi Ota, Yu Jiang, and Daiki Maki. Parameters identification for an inverse problem arising from a binary option using a bayesian inference approach, 2022.

[3] Giovanni S. Alberti, Ernesto De Vito, Matti Lassas, Luca Ratti, and Matteo Santacesaria. Learning the optimal tikhonov regularizer for inverse problems, 2021.

[4] José A Iglesias, Gwenael Mercier, and Otmar Scherzer. A note on convergence of solutions of total variation regularized linear inverse problems. *Inverse Problems*, 34(5):055011, April 2018.

[5] Martin Genzel, Jan Macdonald, and Maximilian Marz. Solving inverse problems with deep neural networks – robustness included? *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(1):1119–1134, January 2023.

[6] Gregory Ongie, Ajil Jalal, Christopher A. Metzler, Richard G. Baraniuk, Alexandros G. Dimakis, and Rebecca Willett. Deep learning techniques for inverse problems in imaging, 2020.

[7] S. V. Venkatakrishnan, C. A. Bouman, and B. Wohlberg. Plug-and-play priors for model based reconstruction. In *IEEE Global Conference on Signal and Information Processing*, pages 945–948, 2013.

[8] S. H. Chan, X. Wang, and O. A. Elgendy. Plug-and-play admm for image restoration: Fixed-point convergence and applications. *IEEE Transactions on Computational Imaging*, 3(1):84–98, 2016.

[9] E. Ryu, J. Liu, S. Wang, X. Chen, Z. Wang, and W. Yin. Plug-and-play methods provably converge with properly trained denoisers. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5546–5557, 09–15 June 2019, June 2019. PMLR.

[10] Alexis Goujon, Sebastian Neumayer, Pakshal Bohra, Stanislas Ducotterd, and Michael Unser. A neural-network-based convex regularizer for inverse problems, 2023.

[11] Amir Beck and Marc Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009.

[12] Y. Romano, M. Elad, and P. Milanfar. The little engine that could: Regularization by denoising (red). *SIAM Journal on Imaging Sciences*, 10(4):1804–1844, 2017.